CRESTRON SIMPL#PRO PROGRAM LIBRARY

# CwsDebug

Self-contained CWS debug log page for Crestron SIMPL#Pro programs

| | |
|---|---|
| **Author** | Mark Jervis |
| **Company** | Stage Front Presentation Systems |
| **Version** | 1.5.1 |
| **Date** | 2026-08-21 |
| **Assembly** | CwsDebug.dll (Crestron.SimplSharp.SDK.ProgramLibrary 2.21.257, net8.0)<br>References Crestron.SimplSharpPro.DeviceSupport |
| **Namespace** | CwsDebug |
| **Public type** | CwsDebugPage |

## Overview

CwsDebug hosts a small, self-contained web page on the Crestron Web Server (CWS) that streams your program's own log messages and — on a hardware appliance — the processor error log. The page provides live search, severity filtering, light/dark themes, a pause control, a master logging switch, and one-click text export. It also hosts a **Signals panel** for viewing and driving a registered device's signals live, in either direction. The entire UI (markup, CSS, and JavaScript) is embedded in the library, so there are no external content files to deploy.

The log is held in a bounded in-memory ring buffer (size configurable at registration) and is **mirrored to disk**, so the most recent entries survive a program restart. Messages are stamped to millisecond resolution, and the page fetches only new entries on each poll. A process-wide **singleton** exposes the whole API statically, so any class on any thread can log without being handed an instance.

> **API note (since 1.1.0):** the API is *static*. Call `CwsDebugPage.LogInfo(…)` and `CwsDebugPage.Register(…)` directly — do *not* construct a `new CwsDebugPage()` (the constructor is private).

It is built as a **Program Library** and referenced by a SIMPL#Pro program; at build time the program bundles `CwsDebug.dll` into its own program archive (`.cpz`). The library is not separately loadable onto a processor.

## Example

The debug page running live on an appliance. The header reads `TEST - Debugger` (the host program is named *TEST*); each entry shows its source, severity badge, message, and timestamp, with search and level filters above.




*CwsDebug page at /cws/debugger/debugger.html (dark theme). Later builds add Logging and Pause controls to the header.*

## Installation

Add a reference to the library from your SIMPL#Pro program project:

```
<ItemGroup>
  <ProjectReference Include="..\CwsDebug\CwsDebug\CwsDebug.csproj" />
</ItemGroup>
```

(Or reference the compiled `CwsDebug.dll` directly.)

## Usage

The API is static, so there is nothing to instantiate. Call `Register()` once from `InitializeSystem()`; you can log from anywhere, on any thread, even before `Register()` runs (early entries are kept and shown once the page is up).

```
using CwsDebug;

public class ControlSystem : CrestronControlSystem
{
    public override void InitializeSystem()
    {
        CwsDebugPage.Register("/debugger");          // default base path; buffer
defaults to 2000 entries
        // CwsDebugPage.Register("/debugger", 5000);  // optional: larger ring buffer
        // CwsDebugPage.MinLevel = CwsDebugPage.DebugLevel.Notice;  // optional: drop
anything below Notice
```

```
        CwsDebugPage.LogInfo("System starting");
        CwsDebugPage.LogNotice("Loaded {0} devices", deviceCount);
        CwsDebugPage.LogWarning("Low buffer on {0}", deviceName);
        CwsDebugPage.LogError("Comms timeout on {0}", deviceName);
        CwsDebugPage.LogFatal("Unrecoverable: {0}", reason);   // also written to the
system ErrorLog
    }

    public override void InitializeSystem_Example_Catch()
    {
        try { /* ... */ }
        catch (Exception e)
        {
            CwsDebugPage.LogException(e, "Error during init");  // full type + message +
stack trace
        }
    }
}
```

Cleanup is automatic — the library releases the CWS server and flushes the log to disk when the program stops or restarts — so calling `Dispose()` / `Shutdown()` by hand is optional.

### Browsing to the page

| Platform | URL |
|---|---|
| Appliance (RMC4, etc.) | `http://<processor>/cws/debugger/debugger.html` |
| VC-4 (Server) | `http://<processor>/VirtualControl/Rooms/<roomId>/cws/debugger.html` |

### API reference

All members are `static` on `CwsDebugPage`.

| Member | Description |
|---|---|
| `LogInfo(msg, args)` | Records an informational message. |
| `LogNotice(msg, args)` | Records a notice-level message. |
| `LogWarning(msg, args)` | Records a warning-level message. *(new in 1.2.0)* |
| `LogError(msg, args)` | Records an error-level message. |
| `LogFatal(msg, args)` | Records a fatal message and mirrors it to the system error log. |

| | |
|---|---|
| `LogException(ex, context?)` | Records an exception at Error level with its full type, message and stack trace (and inner exceptions). *(new in 1.2.0)* |
| `Register(basePath, maxEntries?)` | Starts the CWS server. `basePath` defaults to `"/debugger"`; `maxEntries` defaults to `2000` (values ≤ 0 are ignored). |
| `MinLevel` PROP | Minimum severity recorded ( `DebugLevel` = Info, Notice, Warning, Error, Fatal). Below-floor messages are dropped at the source. Defaults to `Info`. *(new in 1.2.0)* |
| `LoggingEnabled` PROP | Master record switch. When `false`, all `Log*` calls are no-ops (fatals are still mirrored to the system ErrorLog). Persisted across restarts. Defaults to `true`. *(new in 1.2.0)* |
| `Dispose()` / `Shutdown()` | Stops the server, flushes the log, releases resources. Idempotent and optional (cleanup is automatic). |
| `RegisterDevice(name, device, sgdPath?)` | Exposes a device's signals in the Signals panel. Accepts any `BasicTriList` (XPanel, TSW, and other touchpanels). Safe from any thread and before `Register()`. Pass the same SGD path given to `LoadSmartObjects` to have smart objects labelled with their design-time names. *(new in 1.3.0; `sgdPath` in 1.5.0)* |
| `RegisterContract(name, device, cse2jPath)` | As `RegisterDevice`, and additionally loads a Crestron Construct contract so signals can be addressed by their contract item path. If the manifest is missing or unreadable the device still registers and numeric addressing still works, with a warning in the log. *(new in 1.5.0)* |
| `UnregisterDevice(name)` | Drops a registered device; call before disposing it. Returns `false` if the name was not registered. *(new in 1.3.0)* |
| `UnregisterAllDevices()` | Drops every registered device. *(new in 1.3.0)* |
| `ValueWritesAllowed` PROP | Master switch for changing signals, in either direction. When `false` the Signals panel stays read-only whatever the page does. Defaults to `true`. Not persisted. *(new in 1.3.0)* |
| `ValueWritesArmed` PROP | Arm gate for changing signals. Defaults to `false` at every program start and is never persisted. *(new in 1.3.0)* |
| `ValueArmTimeoutMs` PROP | Idle time before the arm gate closes itself. Defaults to `300000` (5 minutes); `≤ 0` disables the timeout. *(new in 1.3.0)* |

Each `Log*` method accepts an optional `string.Format` template plus arguments, e.g. `CwsDebugPage.LogInfo("Connected to {0}:{1}", host, port);`

## Signals panel

Registering a device exposes its signals on the page, where they can be viewed live and driven — the same workflow as Crestron Toolbox, without leaving the browser. One call is all the program needs:

```
public override void InitializeSystem()
{
    CwsDebugPage.Register("/debugger");
    CwsDebugPage.RegisterDevice("Panel", tsPanel);   // any BasicTriList
}
```

Call `UnregisterDevice` before disposing the device. A program's own stop handler usually runs before the library's, so without it the page can briefly reach a disposed panel:

```
CwsDebugPage.UnregisterDevice("Panel");
panel.Dispose();
```

### Adding a signal

Open **Signals** in the header and choose a device, a type (Digital, Analog or Serial) and a direction:

- **In** — signals going *into* the panel: button feedback, gauge levels, indirect text. Driving these changes what the panel *shows*.
- **Out** — signals coming *out of* the panel: presses and slider moves. Driving these makes the program react exactly as if the panel had been used, so they change what the system *does*.
- **Both** — one row showing a signal's two directions together.

> These names follow the panel's point of view, matching Crestron's own naming: *in* to the panel, *out* of the panel. They are the reverse of 1.4.0 and earlier, so a saved row may now carry the opposite label — the signal it refers to has not changed.

Entering a **Smart Object ID** lists that object's signals by name (`Item 1 Pressed`, `Select Item`, `Item 1 Text`) instead of by number, so there is no need to work out which sig number a control uses. The field offers the objects it knows about, so a number or part of a name is enough to find one. Rows can be given friendly names, and the list is remembered in the browser rather than declared in the program.

Smart objects are labelled with their own names when the program supplies them: from the SGD for a Smart Graphics panel (`Button List`, `Simple Keypad`), or from the contract for a Crestron Construct panel (`Main.Dpad`, `Main.HButtonList`), whose signals are then listed by contract name (`Up`, `Down`, `Left` …) rather than as generic numbered signals.

```
tsPanel.LoadSmartObjects(sgdPath);
CwsDebugPage.RegisterDevice("Panel", tsPanel, sgdPath);          // Smart Graphics
CwsDebugPage.RegisterContract("Touchpanel", ch5Panel, "My Project.cse2j");   // Construct
/ CH5
```

A Construct manifest is the `<Project>.cse2j` file from the project's `output\contract\mapping\` folder; the program must ship it as content so it exists on the processor. With it loaded, a signal can also be addressed by its contract item path.

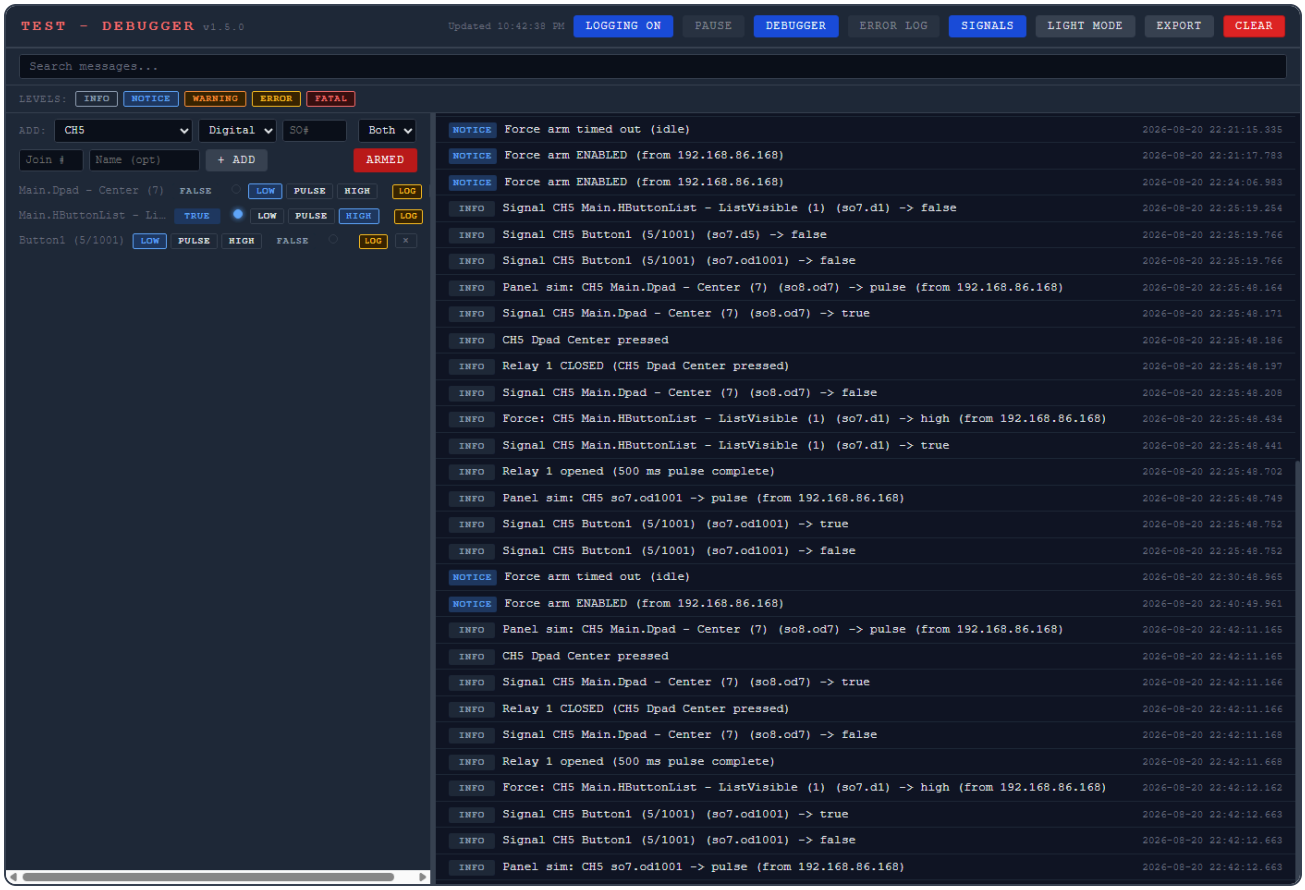
> For a Construct panel, the signal picker only offers joins the manifest actually names. A smart object can expose more raw joins than the contract declares (a Dpad's directions, for instance, are events with no matching state) — those are still reachable by typing a join number directly, just not listed by a name that does not exist.

### Viewing and driving

Digital rows show the signal's state and light a lamp for as long as it is high — a press too brief to fall on a poll still registers as a flash. Analog and serial rows are edited by typing a value and pressing Enter; a value that changes on the system while a field is being edited replaces what was typed, so a force can never overwrite a change it has not shown.

Digital signals are driven with **Low**, **Pulse** and **High**, in both directions. A value is written to the signal itself: unlike Toolbox there is no separate "unforce", so it stands until the program or the panel changes it again.

> **The two directions do different things.** Driving an **In** signal changes what the panel *shows* — a button lights, a gauge moves — and nothing else. Driving an **Out** signal makes the program react as though the panel had been used: its own signal handlers, and Construct contract events, run normally, so the code behind a control can be exercised from the page without touching a panel and without adding anything to the program. Where that is not available on a device the controls are not shown, and the row stays a live read-out.




*The Signals panel beside the log, with digital, analog, serial and Smart Graphics signals. Two rows have **Log** switched on, and the log shows the resulting entries for both directions of a button. The divider between the two panes can be dragged.*

### Logging a signal's changes

Each row also carries a **Log** toggle. With it on, the processor writes a line to the log every time that signal changes:

```
Signal Panel Power Button (od1) -> true
Signal Panel Volume (a1) -> 46536
```

That gives a running trace of any join without adding a single `Log*` call to the program — useful for watching what a panel is actually sending, or confirming what the program drives back. The lines are written on the processor rather than in the browser, so they carry the moment the change happened and include transitions too brief to land on a poll.

- Digital edges are always logged. Analog and serial have a floor between entries, so a dragged slider cannot flood the buffer.
- Entries go through the normal log path, so the master logging switch and the severity floor still apply — unlike a force, which is always recorded.

- Logging a signal stops shortly after the page stops asking for it, so closing the browser cannot leave a processor logging every transition indefinitely.

### Keeping changes safe

- Changing a signal is off until it is armed on the page. Arming is never persisted: it is forgotten whenever the program restarts or the page is reloaded, and lapses after `ValueArmTimeoutMs` (default five minutes) without activity.
- `ValueWritesAllowed = false` makes the panel read-only regardless of the page — the intended setting for a system being handed over, since it keeps the live view.
- Every change and every refusal is written to the log with the address it came from, ignoring both the master logging switch and the severity floor, so the record cannot be turned off.
- Both directions share one arm gate, one kill switch and one audit trail; driving the Out direction runs program logic, so it is never on a lighter footing than an In change.
- Values are never persisted.

### What stops the page updating

Three things are easily confused:

| Control | What it stops |
|---|---|
| **Logging** | Recording on the processor. It does *not* stop the page requesting updates. |
| **Pause** | The page requesting updates. The processor keeps recording; resuming catches up. |
| *(automatic)* | Requests, while the tab is hidden and after fifteen idle minutes. |

Signals refresh about four times a second while the panel is open, so a button press appears in roughly a quarter of a second; the log refreshes every two seconds. Only one request is ever in flight, so a slow connection simply refreshes less often instead of queueing requests. The automatic stop counts arriving values as activity, so watching a panel being pressed keeps the page live even with no mouse or keyboard use; it resumes on a click, a keystroke, or the tab being shown.

## Configuration & persistence

- **Buffer size:** the log is a bounded ring buffer; once full, the oldest line is dropped as each new one arrives. Size it per program with `Register("/debugger", maxEntries)` (default 2000). This bounds both memory and the JSON payload sent to the page.
- **Severity floor (`MinLevel`):** drops sub-threshold messages before they are ever stored — distinct from the page's level-filter buttons, which only hide already-recorded entries in the browser.
- **Master switch (`LoggingEnabled` / "Logging" button):** turns recording on for a debugging session and off when finished. The state is persisted, so it survives a restart. When off, the page

status shows "LOGGING OFF".

- **Persistence:** the buffer is mirrored to the application directory (`cwsdebug_<path>.log`, plus `.state` for the switch) every few seconds when changed and once on program stop, then reloaded on the next `Register()` — so recent history survives a restart.
- **Pause:** a page-side control that freezes polling/rendering. Recording continues on the processor, so resuming shows everything logged during the freeze (subject to the ring cap).

## Platform behavior

- **Base path (VC-4):** with the default base path, the redundant segment is collapsed so the page is served directly at `.../cws/debugger.html`. Passing an explicit (non-default) base path keeps it in the URL on every platform — required if a single program hosts more than one debug page.
- **Concurrent rooms (VC-4):** each room is an isolated program instance with its own CWS namespace and its own log, so multiple rooms run side-by-side without conflict.
- **Header title:** shown as `"{Name} - Debugger"`, where Name is the *room name* on VC-4 and the *host program's namespace/assembly name* on an appliance, falling back to the controller prompt name, then the base-path segment, then "Debugger".
- **Processor error log:** read via the console `err` command and shown only on a hardware appliance; the feature is hidden on VC-4, where no per-program console error log exists.

## A note on network access

The debug page has no login. Anyone who can reach the processor on the network can open the page, read the log, clear it, or switch logging off — the same way Crestron's own built-in web pages work. On a dedicated control-system network this is normally fine. If the processor sits on a network shared with general office traffic or guests, keep in mind that the log is visible to everyone on that network — so avoid logging passwords, door codes, or anything else sensitive, and ask your IT/network administrator to place the processor on a restricted VLAN if that's a concern.

The same applies to the Signals panel, which can change values on a running system. Forcing is switched off until someone arms it, arming is forgotten whenever the program restarts or the page is reloaded, it switches itself back off after five idle minutes, and every force is written to the log with the address of the machine that sent it. For a system being handed over, set `CwsDebugPage.ValueWritesAllowed = false` to leave the panel read-only while keeping the live view.